ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



CO5085 – Học sâu và ứng dụng trong thị giác máy tính

BÁO CÁO ASSIGNMENT 2

Sinh khuôn mặt anime bằng DCGAN
Đối chứng DiffAugment và phân tích paper NeurIPS 2020

Học kỳ 261 – Năm học 2026–2027

Giảng viên: TS. Lê Thành Sách
TS. Nguyễn Quốc Minh
Sinh viên 1: Phạm Duy Anh - 2310139
Sinh viên 2: Nguyễn Phan Tuấn Duy - 2310491

TP. Hồ Chí Minh, Tháng 11 năm 2026

Mục lục

1 Mục tiêu và phạm vi nghiên cứu **2**

2 Dữ liệu, EDA và pipeline **3**
2.1 Nguồn và attribution . . . 3
2.2 Audit, dedup và split . . . 3
2.3 EDA trực quan và quyết định xử lý . . . 4

3 DCGAN và mục tiêu tối ưu **5**
3.1 Generator và discriminator . . . 5
3.2 Loss và hai pha cập nhật . . . 6

4 Phân tích paper DiffAugment **7**
4.1 Động cơ, phương pháp và thí nghiệm gốc . . . 7
4.2 Diễn giải đường gradient từ implementation đang dùng . . . 7
4.3 Phân tích lựa chọn policy và giả thiết . . . 8
4.4 Đánh giá thiết kế thực nghiệm và giới hạn chuyển giao . . . 8

5 Recipe, điều phối và EMA **9**
5.1 EMA và checkpoint dùng đánh giá . . . 9
5.2 Môi trường và định nghĩa thời gian . . . 9

6 Protocol đánh giá tạo sinh **10**
6.1 Không gian đặc trưng và mẫu tham chiếu . . . 10
6.2 FID, KID và Inception Score . . . 10
6.3 Precision, recall và benchmark . . . 11

7 Kết quả định lượng **11**

8 Learning curves và kiểm tra định tính **12**
8.1 Loss trong quá trình train . . . 12
8.2 Samples từ final EMA: baseline . . . 13
8.3 Samples từ final EMA: DiffAugment . . . 14
8.4 Nội suy latent . . . 15

9 Nearest neighbors và memorization **16**

10 Hạn chế, kết luận và tái lập **17**
10.1 Tái lập và kiểm toán . . . 17

A Nearest neighbors của baseline **19**

1 Mục tiêu và phạm vi nghiên cứu

Bài toán là sinh ảnh khuôn mặt anime RGB64 × 64 từ latent ngẫu nhiên, không dùng nhãn điều kiện. Hai biến thể có cùng generator/discriminator DCGAN, khác policy augmentation trong đường huấn luyện: baseline không DiffAugment, biến thể còn lại dùng color, translation và cutout. Cùng seed 42, cùng split, khởi tạo, optimizer, EMA và ngân sách 20.000 updates giúp phép so sánh tập trung vào thay đổi này.

Báo cáo gồm phân tích paper DiffAugment [9] và thực nghiệm độc lập trên Anime Face Dataset. Đây là áp dụng một ý tưởng phương pháp trong bối cảnh DCGAN/Anime, *không* phải tái lập toàn bộ paper. Các số đo của paper được tách rõ khỏi kết quả nhóm đã lưu; không so trực tiếp FID khác dataset/extractor/protocol như cùng thang chất lượng tuyệt đối.

Bảng 1: Truy vết yêu cầu từ `a2_co5085.md`.

Yêu cầu / bằng chứng	Phần
Bài toán tạo sinh, dataset và EDA	2
Ít nhất hai phương pháp/biến thể có đối chứng	3, 5
Paper: động cơ, phương pháp, thí nghiệm và hạn chế	4
Protocol FID/KID/IS/precision/recall và chi phí	6
Bảng kết quả, loss, samples và nội suy	7–8
Nearest neighbors, giới hạn và tái lập	9, 10, 10.1
Video thuyết trình	Chưa cung cấp

DiffAugment cải thiện FID từ 298,51 xuống 41,84; recall tăng từ 0 lên 0,0584 nhưng vẫn thấp. Vì vậy kết luận là cải thiện chất lượng và độ phủ *tương đối với baseline*, không phải đã bao phủ tốt toàn phân phối anime.

Khai báo sử dụng AI

AI được sử dụng để hỗ trợ thiết kế giao diện website và trực quan hóa kết quả; nghiên cứu tài liệu, đối chiếu nguồn tham khảo; xây dựng, rà soát và debug mã nguồn; thiết kế cơ chế điều phối huấn luyện song song bằng hai tiến trình trên hai GPU; tổ chức artifact, kiểm tra tính nhất quán và soạn thảo báo cáo LaTeX. Các số đo trong báo cáo được lấy từ artifact và notebook đã chạy, không thay bằng kết quả do AI tự tạo. Hỗ trợ về mã điều phối không đồng nghĩa AI đã thực hiện thêm các lượt huấn luyện mới trong quá trình biên soạn báo cáo. Hai thành viên chịu trách nhiệm về nội dung và kết quả được trình bày; cần tự rà soát nguồn, kiểm tra mã, hiểu phương pháp và có khả năng giải thích, bảo vệ toàn bộ nội dung trước khi nộp. Việc dùng AI không thay thế các yêu cầu tự thực hiện và hiểu bài trong đề. URL video chưa được cung cấp và không được tự bổ sung.

2 Dữ liệu, EDA và pipeline

2.1 Nguồn và attribution

Nguồn mount của study là Kaggle `splcher/animefacedataset` [7]; manifest ghi attribution tới repository Anime-Face-Dataset [1]. Không suy ra quyền tái phân phối hình gốc chỉ từ việc tải được dataset. License note trong manifest yêu cầu kiểm tra metadata Kaggle và quyền từ nguồn gốc; báo cáo không tự gán giấy phép chuẩn. Bộ báo cáo chỉ kèm hình minh họa cần cho phân tích, không đóng gói raw dataset.

2.2 Audit, dedup và split

Code đọc từng ảnh, chuyển RGB, lấy kích thước và hash `SHA256(str(size) + RGBpixels)` trước split. Do đó dedup dựa trên pixel đã giải mã kèm shape, không chỉ hash byte file. Tệp khác encoding nhưng cùng ảnh RGB có thể bị gộp; near-duplicate hoặc cùng nhân vật ở crop khác không được chứng minh đã loại. Split ngẫu nhiên seed 42 sau dedup:34.522 train,2.000 validation

Bảng 2: EDA từ manifest: dedup theo SHA-256 (shape + pixel RGB) trước khi split, không phải kiểm tra near-duplicate.

Thuộc tính	Giá trị
Tệp đã kiểm tra	63,565
Trùng hash pixel đã loại	22,043
Tệp lỗi	0
Ảnh duy nhất	41,522
Train / val / test	34,522 / 2,000 / 5,000
Chiều rộng min–max	25–220
Chiều cao min–max	25–220

và 5.000 test. Manifest hash:

```
3cb3d13301f9d37539352fd2f9a0ba3bc31d61706612d14870908adb4ebb202d
```

Không có nhãn lớp để bảo đảm phân tầng theo nhân vật, giới tính hoặc phong cách. Số lượng ảnh duy nhất không tương đương số nhân vật hoặc số mode độc lập. Đây là hạn chế khi đánh giá coverage.

Khác A1 giữ split official và công bố duplicate, A2 dedup *trước* chia tập theo protocol đã chọn. Không nhập hai chính sách thành một mô tả chung. Các con số trên tính từ manifest đã xuất, không phải kiểm tra lại raw images trong lần biên soạn này.

2.3 EDA trực quan và quyết định xử lý



Hình 1: Grid train từ output EDA của notebook A2 đã chạy; mẫu dùng để quan sát miền ảnh và preprocessing.

Ảnh có màu tóc, mắt, nền và góc nhìn đa dạng, nhưng đều tập trung vào khuôn mặt crop. EDA gợi ý cần giữ thông tin màu và hình dạng khuôn mặt khi đưa về kích thước thống nhất; không chuyển grayscale. Pipeline dùng `ImageOps.fit(..., (64,64), BILINEAR)`: resize giữ tỷ lệ rồi crop để khớp khung vuông, không chỉ kéo giãn hai chiều độc lập. Việc crop vẫn có thể mất tóc/nền hoặc phần mặt ở ảnh lệch tâm.

Ảnh cache dạng uint8, loader đổi sang tensor CHW trong$[-1, 1]$ bằng $x/127{,}5 - 1$ để khớp Tanh generator. Train có horizontal flip xác định theo hash(seed,epoch,index), chung cho cả hai biến thể. Validation/test không flip. Vì vậy "baseline không augmentation" trong so sánh này phải hiểu là *không DiffAugment*; nó vẫn có preprocessing và horizontal flip chung.

DiffAugment là can thiệp riêng trong đường vào discriminator, không được ghi đè lên file/cache ảnh train và không áp dụng khi tính metric trên mẫu sinh cuối. Policy là giả thuyết cần đo; việc đổi màu/cutout có thể gây hại nếu phá tín hiệu miền dữ liệu, nên giữ đối chứng baseline là cần thiết.

3 DCGAN và mục tiêu tối ưu

3.1 Generator và discriminator

DCGAN [6] dùng convolution để ánh xạ latent sang ảnh và phân biệt real/fake. Study này có latent $z \sim \mathcal{N}(0, I_{128})$, width 64. Generator nhận$B \times 128 \times 1 \times 1$; chuỗi ConvTranspose2d tạo$4 \to 8 \to 16 \to 32 \to 64$pixel. Các tầng ẩn có BatchNorm+ReLU, đầu ra Tanh. Generator có 3.806.080 tham số theo metrics (không phải tổng G+D).

Bảng 3: Kiến trúc triển khai; mọi convolution dùng bias=False.

Mạng	Kênh / thao tác	Kích thước không gian
G	$128 \to 512$, kernel 4 stride 1 padding 0	$1 \to 4$
G	$512 \to 256 \to 128 \to 64$, kernel 4 stride 2 padding 1	$4 \to 8 \to 16 \to 32$
G	$64 \to 3$, kernel 4 stride 2 padding 1, Tanh	$32 \to 64$
D	$3 \to 64$, Conv 4 stride 2, LeakyReLU 0,2	$64 \to 32$
D	$64 \to 128 \to 256 \to 512$, Conv 4 stride 2, BN, LeakyReLU	$32 \to 16 \to 8 \to 4$
D	$512 \to 1$, Conv 4 stride 1 padding 0	$4 \to 1$

D trả logits thực, không gắn Sigmoid trong module; BCEWithLogits xử lý ổn định số học. Khởi tạo Conv/ConvTranspose bằng $\mathcal{N}(0, 0{,}02^2)$, BatchNorm weight$\mathcal{N}(1, 0{,}02^2)$ và bias 0. Không dùng pretrained trong hai biến thể.

```
# Generator core, with w=64 and z=128:
for a,b,k,s,p in [(z,8*w,4,1,0), (8*w,4*w,4,2,1),
                  (4*w,2*w,4,2,1), (2*w,w,4,2,1)]:
    layers += [nn.ConvTranspose2d(a,b,k,s,p,bias=False),
               nn.BatchNorm2d(b), nn.ReLU(True)]
layers += [nn.ConvTranspose2d(w,3,4,2,1,bias=False), nn.Tanh()]
```

3.2 Loss và hai pha cập nhật

Đặt $s(u) = \log(1 + e^u)$, $D(x)$ là logit, T là identity ở baseline hoặc DiffAugment ở biến thể cải tiến. Code hiện tại tương ứng loss non-saturating:

$$L_D = \mathbb{E}_x[s(-D(T(x)))] + \mathbb{E}_z[s(D(T(G(z))))], \tag{1}$$

$$L_G = \mathbb{E}_z[s(-D(T(G(z))))]. \tag{2}$$

Ở pha D, fake được tạo trong `no_grad()` và tách khỏi graphG; real/fake được ghép rồi augmentation. Ở pha G, tạo latent mới, G giữ graph, D chuyển `eval()` và `requires_grad_(False)`. Đóng gradient tham sốD không ngăn gradient theo inputD đi vềG; nếu gọi `no_grad()` quanh D trong pha G thì đường học này sẽ hỏng.

```
# D phase: real and detached fake are augmented.
x, aug_state = augment(torch.cat([real, fake.detach()]), name,
                       aug_state)
a, b = d(x).chunk(2)
loss_d = BCEWithLogits(a, ones) + BCEWithLogits(b, zeros)

# G phase: no detach; keep input gradients through D and T.
d.eval().requires_grad_(False)
fake = g(z_new)
augmented, next_aug = augment(fake, name, aug_state)
loss_g = BCEWithLogits(d(augmented), ones)
```

D dùng eval trong pha G để không cập nhật BatchNorm buffers ở bước tối ưuG; G vẫn ở train khi cập nhật. Mô tả này là quyết định implementation của study, không khẳng định mọi DCGAN chuẩn đều dùng đúng cùng quy ước.

Hai loss là hai mục tiêu đang thay đổi lẫn nhau, nên không đọc như supervised loss một mô hình cố định. GLoss thấp có thể xuất hiện dù samples lặp và FIDxấu. Vì vậy phải đọc cùng metric phân phối, grid và nearest neighbors thay vì chọn run chỉ theo lossnhỏ nhất.

4 Phân tích paper DiffAugment

4.1 Động cơ, phương pháp và thí nghiệm gốc

Tóm tắt từ paper [9], Phần 3–4: Ít dữ liệu khiến discriminator dễ nhớ train. DiffAugment dùng biến đổi khả vi cho real/fake trong cả hai loss. Chỉ augment real làm lệch đích; augment ở D nhưng bỏ qua G làm mục tiêu bất cân xứng. Paper thử BigGAN/StyleGAN2 trên ImageNet, CIFAR, FFHQ/LSUN và low-shot. Trên ImageNet 128, BigGAN+DiffAugment đạt FID 6,80, IS 100,8 (50.000 mẫu, chọn snapshot FID tốt nhất, ba lần evaluation). Đây là số của tác giả paper, không phải DCGAN/Anime. Study này chỉ áp dụng policy, không tái lập toàn bộ thí nghiệm gốc.

4.2 Diễn giải đường gradient từ implementation đang dùng

Trong code A2, T là chuỗi color, translation, cutout [8]. Với một realization của ngẫu nhiên ξ, đường lan truyền ở pha G là

$$\nabla_{\theta_G} L_G = \frac{\partial L_G}{\partial D}\frac{\partial D}{\partial T}\frac{\partial T_\xi}{\partial G}\frac{\partial G}{\partial \theta_G}. \tag{3}$$

Điều kiện cần ở đây là đạo hàm theo *giá trị pixel đầu vào*; không cần lấy gradient theo chỉ số dịch nguyên hoặc vị trí mask ngẫu nhiên. Gather/indexing chuyển gradient tới pixel được chọn; cutout nhân mask 0/1 cho gradient 0 ở vùng che và giữ đường ở phần còn lại. Nếu đổi ảnh sang PIL/NumPy giữa G và D thì autograd bị cắt, dù hình sau augmentation trông đúng.

Cùng policy không có nghĩa mọi ảnh real/fake phải nhận cùng một realization pixel. Code ghép batch rồi các hàm lấy ngẫu nhiên theo sample; pha G có phép lấy ngẫu nhiên tiếp theo. RNG augmentation được lưu riêng để latent/model RNG không chi phối một cách không kiểm soát. Baseline trả input nguyên dạng tại hàm này.

Đây là giải thích toán học của pipeline triển khai, không lấy kết quả số của paper để chứng minh correctness. Kiểm thử code, recipe đã khóa và so sánh control mới là bằng chứng cho hệ thống hiện có.

4.3 Phân tích lựa chọn policy và giả thiết

Color trong code gồm brightness, saturation và contrast. Brightness cộng hằng ngẫu nhiên mỗi ảnh; saturation nội suy quanh mean theo kênh; contrast co giãn quanh mean toàn ảnh. Các phép tính tensor giữ đạo hàm theo pixel nhưng có thể tạo giá trị ra ngoài miền ảnh ban đầu khi train. Không clamp/uint8 trong đường này; quantize chỉ thuộc protocol metric ở bước khác.

Translation lấy dịch nguyên trong phạm vi khoảng 1/8 kích thước, padding 0; cutout mask vùng khoảng nửa cạnh ảnh. Cả hai tăng biến thiên mà không cần bổ sung dữ liệu thật. Đổi lại, chúng có thể che bộ phận khuôn mặt quan trọng. Việc cả real và fake được xét sau T giảm khả năng D học một phía bằng dấu vết augmentation, nhưng không bảo đảm mọi thông tin về phân phối gốc vẫn được phân biệt.

Giả thiết làm việc của study là T giữ đủ tín hiệu miền khuôn mặt để D vẫn cung cấp gradient hữu ích. Nếu mask quá mạnh hoặc color biến đổi quá mức, D có thể học ít về cấu trúc thật. Vì không tách từng policy trong ablation, kết quả hiện tại chỉ gán cho tổ hợp `color,translation,cutout`, không nói riêng cutout gây phần lớn cải thiện.

4.4 Đánh giá thiết kế thực nghiệm và giới hạn chuyển giao

Thí nghiệm paper có model, dataset, resolution, cách chọn snapshot và số mẫu đánh giá khác study này. Do đó không dùng FID 6,80 của paper để làm mục tiêu nghiệm thu FID Anime, cũng không mô tả FID 41,84 như "kém paper" theo một phép xếp hạng trực tiếp. Muốn tái lập phải khớp đầy đủ code, data, budget, preprocessing và metric protocol.

Điểm hữu ích của ý tưởng là có thể kiểm chứng ở DCGAN nhỏ bằng cách giữ architecture và recipe đối chứng. Tuy nhiên một policy tốt trên Anime chưa chứng minh hiệu quả ở nhiều miền dữ liệu hoặc chế độ cực ít ảnh. Nghiên cứu hiện dùng 34.522 train images, không phải setup 100-shot. Nhận xét giới hạn này là đánh giá của báo cáo về phạm vi suy luận, không gán thành tuyên bố mới của paper.

Một ablation tiếp theo nên thử color/translation/cutout riêng, nhiều seed, số lượng train khác nhau và giữ ngân sách update rõ ràng. Các thí nghiệm đó chưa chạy nên không điền giả số vào bảng.

5 Recipe, điều phối và EMA

Hai variant dùng Adam cho cả G và D, LR$2 \cdot 10^{-4}$, betas$(0{,}5, 0{,}999)$; batch 64 và microbatch 64 nên không tích lũy gradient trong cấu hình thực. Mỗi update hoàn thành một pha D và một pha G;20.000 updates tương ứng 1.280.000 lượt ảnh thật đưa vào, không phải 1.280.000 ảnh duy nhất hay 20.000 epoch.

Dataloader shuffle bằng generator seed+epoch,2 workers và `drop_last=True`; horizontal flip theo sample/epoch giữ chung hai variant. Hai worker độc lập trên hai GPU, không train bằng threads trong cùng process. Checkpoint lưu G, D, EMA, hai optimizer/scaler, latent RNG, augmentation RNG, RNG toàn cục, step/epoch/offset/history và fingerprint. Resume tại update boundary yêu cầu identity khớp, không ghép run khác split/config.

AMP dùng loss scale ban đầu 1024. Cơ chế tối ưu kiểm tra nonfinite và có retry tối đa 8 lần cho phase bị tràn, phục hồi buffers/RNG cần thiết trước replay. Update chỉ được tính khi pha thành công; nonfinite forward loss là lỗi cứng. Báo cáo không coi một attempt overflow là một update hoàn tất.

5.1 EMA và checkpoint dùng đánh giá

EMA decay 0,999 cập nhật sau mỗi cặp D/G thành công:

$$\theta_{\text{EMA}} \leftarrow 0{,}999\theta_{\text{EMA}} + 0{,}001\theta_G. \tag{4}$$

Implementation copy buffers trực tiếp từ G, không lấy trung bình running statistics theo công thức EMA tham số. Final evaluation dùng G EMA ở update 20.000, không quét test FID để chọn checkpoint tốt nhất. `validation_real_score` trong history là mean Sigmoid D trên một batch real validation định kỳ, không phải validation FID hay accuracy đầy đủ.

5.2 Môi trường và định nghĩa thời gian

Python 3.13.16, PyTorch 2.11.0+cu130, CUDA13.0, NumPy 2.1.3, SciPy 1.16.3, Pillow 13.0.0, torch-fidelity 0.3.0, Tesla T4. Code tính elapsed wall time vòng train, gồm data/compute và overhead định kỳ checkpoint/preview/validation trong vòng lặp, không gồm final metric extraction. Khác A1, không khẳng định thời gian A2 đã loại toàn bộ I/O. Hai worker cùng host cũng chia sẻ CPU/I/O, nên số đo không mô tả GPU cô lập.

6 Protocol đánh giá tạo sinh

6.1 Không gian đặc trưng và mẫu tham chiếu

Cả hai biến thể dùng `inception-v3-compat` của torch-fidelity 0.3.0 [5], feature 2048 và logits unbiased. Ảnh$[-1, 1]$ được clamp, scale$[0, 255]$, round và cast RGB uint8 trước extractor. Đánh giá FP32 với 5.000 generated và 5.000 test của split đã đăng ký; metric seed 2068. Đây không phải encoder được huấn luyện riêng trên anime, nên embedding có thể bỏ sót tính chất quan trọng với người xem.

Hash weights extractor:

```
2037b63fbc78aaa2ec9b4d735247282ba0ee344d1ba7166922951fe344f51806
```

Protocol còn ghi source hash, manifest, batch 64, KID 100 subsets×1.000 mẫu, IS 10 splits, kNN k=3. Hash giúp ngăn so sánh metrics khi đã đổi extractor/split không báo.

6.2 FID, KID và Inception Score

FID [3] xấp xỉ hai phân bố feature bằng Gaussian:

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r^{1/2}\Sigma_g\Sigma_r^{1/2})^{1/2}). \tag{5}$$

Code dùng float64 và eigendecomposition cho covariance. FID thấp hơn là tốt hơn trong cùng protocol, nhưng 5.000 mẫu vẫn là ước lượng hữu hạn; không diễn giải như khoảng cách chính xác của hai phân bố thật.

KID [2] là MMD^2 ước lượng không chệch với kernel bậc 3 $k(x, y) = (x^\top y/d + 1)^3$, lấy trung bình 100 subset độ lớn 1.000. Std trong bảng là độ phân tán các subset score từ cùng checkpoint, không phải biến thiên giữa 100 lần training. KID giữ nguyên thang scale 1.

IS được tính từ logits của ảnh sinh: $\exp(\mathbb{E}_x \mathrm{KL}(p(y|x)\|p(y)))$. Logits ImageNet không đồng nghĩa nhãn nhân vật anime; IS cao chưa bảo đảm giống data hoặc không memorize. Std ở đây là std 10 splits của cùng 5.000 ảnh, không phải multi-seed uncertainty.

6.3 Precision, recall và benchmark

Precision/recall [4] dùng các lân cận feature với bán kính đến kNN thứ 3. Precision đo tỷ lệ mẫu sinh nằm trong manifold ước lượng từ real; recall đo tỷ lệ real được manifold generated bao phủ. Chúng là độ đo trong embedding, không phải phần trăm "ảnh đúng" hay độ phủ mọi thuộc tính mặt anime.

Một generator có thể tạo một vài kiểu ảnh khá thuyết phục, đạt precision tương đối cao nhưng recall rất thấp. Vì vậy phải đọc hai số cùng FID và samples. Recall baseline 0 là một giá trị đo thật, không phải file thiếu hoặc lỗi phép chia không.

Benchmark generator EMA FP32 trên Tesla T4, batch 1 và 64, warmup 5 và 30 lần đo, có CUDA synchronize. Latency chia số ảnh, throughput là ảnh/giây, memory là peak allocated MiB. Số tham số trong metrics là G; discriminator chỉ dùng training nên không được tính vào chi phí inference generator.

7 Kết quả định lượng

Bảng 4: Đánh giá final EMA; KID ở thang gốc (không nhân 100 hay 1000).

Biến thể	FID↓	KID↓	KID std	IS↑	IS std
Baseline	298.51	0.38419	0.00160	1.347	0.010
+ DiffAugment	41.84	0.03749	0.00131	2.000	0.071

Bảng 5: Độ phủ trong không gian Inception và chi phí training.

Biến thể	Precision%	Recall%	Train (s)	VRAM (MiB)
Baseline	52.90	0.00	921.84	210.64
+ DiffAugment	59.38	5.84	928.05	210.64

Bảng 6: Benchmark generator EMA FP32, 30 lần đo trên Tesla T4.

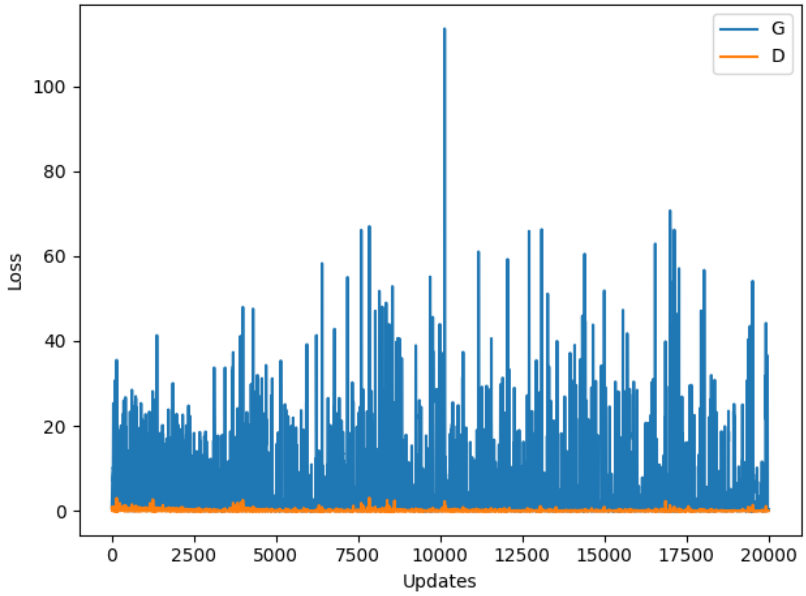
Biến thể	Batch	ms/ảnh	ảnh/s	MiB
Baseline	1	0.989	1010.7	15.03
Baseline	64	0.277	3614.5	100.35
+ DiffAugment	1	0.947	1055.7	15.03
+ DiffAugment	64	0.286	3495.9	100.35

Cùng protocol, DiffAugment giảm FID 256,67 điểm (khoảng 85,98% so baseline), giảm KID và tăng IS. Precision tăng từ 52,90% lên 59,38%; recall tăng từ 0 lên 5,84% nhưng vẫn rất thấp. Không kết luận đã bao phủ tốt toàn dataset. Chi phí train 928,05 s so 921,84 s trong hai queue đồng thời; độ chênh lệch nhỏ không phải bằng chứng overhead cố định trên mọi thiết bị.

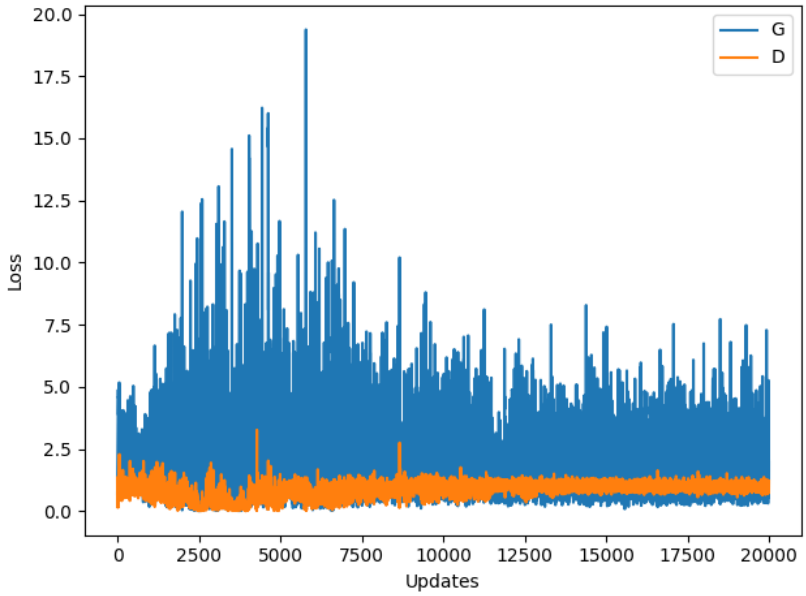
Các tham số generator giống nhau nên không có lợi ích capacity từ việc thêm DiffAugment vào inference. Chênh lệch latency nhỏ giữa hai weights được xem là dao động đo; augmentation chỉ tham gia train.

8 Learning curves và kiểm tra định tính

8.1 Loss trong quá trình train



Hình 2: Baseline: G/D loss theo 20.000 updates, không phải supervised validation curve.



Hình 3: DiffAugment: cùng ngân sách update và architecture.

Ở cuối baseline, G loss 0,2137 và D loss 0,0013 trong history; DiffAugment tương ứng 1,9644 và 1,2106. Dù loss G baseline thấp hơn, metric và grid cho chất lượng kém hơn. Loss GAN phụ thuộc trạng thái đối thủ và mode BatchNorm, không có quy tắc "G loss thấp nhất là ảnh đẹp nhất". `validation_real_score` chỉ là mean D trên một batch real, không đủ để chứng minh coverage hay chọn model.

8.2 Samples từ final EMA: baseline



Hình 4: 64 mẫu baseline, latent grid cố định từ export; không chọn lại mẫu trong lần báo cáo.

Baseline lặp mạnh khuôn mặt, tóc và nền gần nhau, phù hợp với dấu hiệu mode collapse và recall 0. Dù có hình dạng khuôn mặt nhận biết được, generator ít thay đổi kiểu tóc, bố cục và nền khi đổi latent trong grid này. Precision 52,90% không mâu thuẫn với hiện tượng đó: một số kiểu ảnh có thể gần manifold real nhưng không phủ được các vùng khác.

Đây là quan sát trong grid, không phải đếm coverage định lượng của toàn 5.000 ảnh. Kết luận coverage phải đọc từ recall với cùng extractor, đồng thời giữ giới hạn domain của Inception. Loss G thấp ở cuối train chưa phản ánh được vấn đề lặp mẫu nhìn thấy ở đây.

8.3 Samples từ final EMA: DiffAugment



Hình 5: 64 mẫu DiffAugment từ cùng quy ước latent grid.

DiffAugment có nhiều màu tóc, khuôn mặt và góc nhìn hơn baseline trong grid đã lưu; một số mắt/đường viền vẫn biến dạng. FID giảm mạnh được củng cố bởi quan sát thị giác này, nhưng recall 5,84% vẫn hạn chế kết luận đa dạng. Không gọi các mẫu này là đã bao phủ tốt toàn phân phối dữ liệu.

Hai grid dùng cùng quy ước latent cố định để hỗ trợ đối chiếu; không thay bằng các ảnh được chọn thủ công vì đẹp hơn. Màu tóc đa dạng chưa chứng minh đa dạng theo nhân vật, giới tính hoặc phong cách vì dataset không có nhãn thuộc tính để kiểm chứng các chiều đó.

8.4 Nội suy latent



Hình 6: Baseline:8 điểm nội suy tuyến tính giữa hai latent chuẩn.

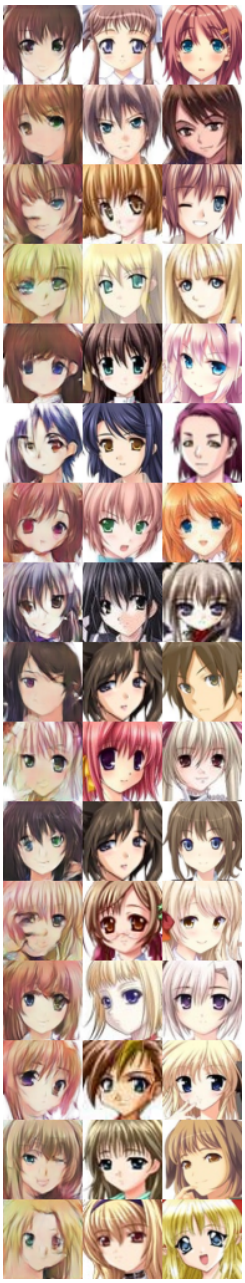


Hình 7: DiffAugment: cùng cách dựng 8 điểm, dùng EMA final.

Code chọn hai vector latent ngẫu nhiên và tính$z_\alpha = (1-\alpha)z_0 + \alpha z_1$ với 8 giá trị đều trong$[0, 1]$. Một đường nội suy có thay đổi mượt cho thấy G liên tục ở đoạn đó, không chứng minh latent disentanglement hoặc phủ toàn miền. Baseline có thể liên tục nhưng vẫn lặp một type; DiffAugment biến thiên thị giác tốt hơn trong grid đã lưu nhưng chưa có thí nghiệm thuộc tính độc lập.

Các ảnh nội suy không phải ảnh trích dataset. Không dùng chúng như thêm 5.000 mẫu metric, không gộp với random samples để tăng chất lượng đánh giá. Tính định lượng vẫn dùng protocol cố định ở Phần 6.

9 Nearest neighbors và memorization



Hình 8: Mỗi hàng: generated, nearest train, nearest test trong feature Inception;16 ảnh sinh đầu tiên, DiffAugment.

Đây là chẩn đoán trong không gian feature, không phải tìm theo pixel. Code giới hạn pool train tham chiếu ở 5.000 ảnh đầu được extract, test 5.000 ảnh; vì vậy không gọi là nearest neighbor trên toàn bộ 34.522 train. Chỉ 16 ảnh sinh được hiển thị. Có ảnh gần nhau không đủ kết luận memorization; không thấy bản sao cũng không chứng minh không memorize.

Hướng kiểm tra tiếp là tìm lân cận trên toàn bộ train và dùng thêm pixel/perceptual distance với ngưỡng đăng ký trước. Near-duplicate khác crop vẫn có thể qua exact dedup; các thí nghiệm đó chưa được thực hiện trong báo cáo này.

10 Hạn chế, kết luận và tái lập

Đây là study hai biến thể DCGAN, không so sánh GAN với diffusion và không tái lập BigGAN/StyleGAN2 của paper. Cùng seed không loại bỏ tính ngẫu nhiên hoặc ước lượng được phương sai qua lần train. KID subset std và IS split std không phải khoảng tin cậy multi-seed; chưa có kiểm định ý nghĩa thống kê.

Extractor ImageNet có thể không bắt hết sai lệch khuôn mặt anime; số mẫu 5.000 hữu hạn. Exact dedup không loại near-duplicate; split không phân tầng theo nhân vật. Policy gộp ba thành phần nên chưa quy kết được tác động riêng. Final EMA được cố định ở 20.000 updates, không chứng minh hội tụ. Grid/nearest neighbor chỉ là chẩn đoán, không bảo chứng không sao chép dữ liệu.

Kết luận được hỗ trợ là DiffAugment cải thiện mạnh so với baseline ở recipe này, nhưng recall còn thấp và lỗi thị giác vẫn có. Bước tiếp nên lặp nhiều seed, ablate policy, tăng độ chặt chẽ của kiểm tra memorization và đánh giá human perception, thay vì chỉ tối ưu FID. Không cộng các kế hoạch đó vào kết quả đã đạt.

10.1 Tái lập và kiểm toán

Notebook nguồn `A2/a2_dcgan_diffaugment.ipynb`; notebook đã chạy `A2/dcgan-vs-augmenteddcgan-dl-assignment-2.ipynb`. Bundle `artifacts/A2/` giữ config, environment, metric protocol, full history 20.000 dòng/variant và hình. Website gộp history để hiển thị; bảng báo cáo dùng artifact gốc, không dùng preview hoặc nội suy dữ liệu.

`pythontools/prepare_reports.py` kiểm tra hash bundle và ngân sách, sinh bảng/hình và provenance. Source validation checkpoint được ghi ở lần pack gốc; không mô tả như lần biên soạn này nạp checkpoint đã loại khỏi bundle. `pythontools/build_reports.py` build PDF không cần ML libraries/GPU/raw dataset. Video và xác nhận các mốc nộp bài vẫn là phần cần hoàn thiện trước khi nộp.

Tài liệu

[1] bchao1. Anime-face-dataset. https://github.com/bchao1/Anime-Face-Dataset. Attribution được ghi trong manifest.

[2] Mikołaj Bińkowski et al. Demystifying mmd gans. https://arxiv.org/abs/1801.01401, 2018.

[3] Martin Heusel et al. Gans trained by a two time-scale update rule converge to a local nash equilibrium. https://arxiv.org/abs/1706.08500, 2017.

[4] Tuomas Kynkäänniemi et al. Improved precision and recall metric for assessing generative models. https://arxiv.org/abs/1904.06991, 2019.

[5] Anton Obukhov et al. torch-fidelity: reproducible generative model evaluation. https://github.com/toshas/torch-fidelity. Extractor sử dụng phiên bản 0.3.0.

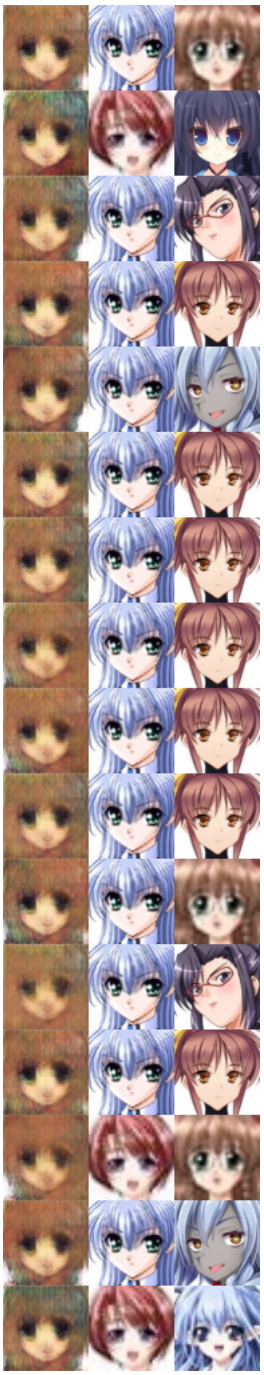
[6] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. https://arxiv.org/abs/1511.06434, 2015.

[7] splcher. Anime face dataset. https://www.kaggle.com/datasets/splcher/animefacedataset. Nguồn mount Kaggle của thí nghiệm.

[8] Shengyu Zhao and collaborators. Data-efficient gans: official implementation. https://github.com/mit-han-lab/data-efficient-gans, 2020.

[9] Shengyu Zhao, Zhijian Liu, Ji Lin, Jun-Yan Zhu, and Song Han. Differentiable augmentation for data-efficient gan training. In *NeurIPS*, 2020. https://arxiv.org/abs/2006.10738.

A Nearest neighbors của baseline



Hình 9: Baseline: generated/nearest train/nearest test với cùng giới hạn pool tham chiếu. Lặp mẫu quan sát được phải xem cùng recall 0 và FID 298,51.